

SMART INPUT HUB

Dossier de Arquitectura, Especificación de Producto y Planos de Interfaz

Preparado para: Federico Verges | **Enfoque Estratégico:** Local-First (Fase 1) a Cloud Premium (Fase 2) | **Fecha de Emisión:** Junio 2026

1. Resumen Ejecutivo y Filosofía de Evolución

Este informe técnico consolida la estrategia de producto resultante de fusionar las propuestas de diseño originales basadas en tarjetas de esquinas suavizadas y paleta cromática sutil, junto con el análisis de mercado del competidor local de pago único (*GuitaFix*).

El pilar fundamental de esta arquitectura es el enfoque **Local-First Estratégico**. Al posponer temporalmente la capa de procesamiento en la nube (NLP/OCR y sincronización distribuida) para una versión posterior, se elimina por completo la latencia de red en el uso diario, se garantizan costos operativos de infraestructura cero (\$0) durante el lanzamiento, y se blinda la privacidad absoluta del usuario al almacenar los movimientos financieros estrictamente dentro de la memoria física del dispositivo.

2. Matriz de Fases y Roadmap del MVP

La aplicación se construirá bajo un esquema iterativo dividido en dos grandes fases de ingeniería, permitiendo llegar al usuario final (Play Store) de forma ágil mediante un producto mínimo viable altamente funcional.

Área Funcional	Fase 1: MVP (Local-First Offline)	Fase 2: Premium (Cloud Connected)
Almacenamiento	Base de datos local segura (SQLite / MMKV en dispositivo).	Sincronización bidireccional asíncrona con la nube (PostgreSQL/Supabase).
Carga de Datos	Formulario manual de alto rendimiento (foco automático, menos de 3 clics).	Inferencia multimodal mediante OCR de tickets y análisis de lenguaje natural en backend.
Finanzas Especiales	Módulos de <i>Gastos Fijos</i> mensuales y control diferido de cuotas (<i>CuotaFix</i>) locales.	Alertas de vencimiento automatizadas mediante notificaciones push y flujos compartidos.
Soporte Divisas	Patrimonio bimonetario consolidado con actualización manual de cotización.	Conexión automática a APIs del mercado financiero (Dólar Blue, MEP, Cripto y CEDEARs).

Justificación del Cambio de Carga Manual

Al remover el procesamiento lingüístico complejo del MVP, el diseño de la interfaz se optimizó para acelerar la carga analógica. El Botón Flotante (FAB) despliega instantáneamente un teclado numérico dedicado, reduciendo la entrada de un gasto a un patrón táctil repetitivo que toma menos de 4 segundos en completarse.

3. Especificación de Diseño de Interfaz (Planos UX)

A continuación se presentan los planos estructurales que representan fielmente los diseños creados, aplicando la nueva paleta de colores sutiles basada en grises limpios y el color turquesa empolvado como eje de acento visual.

Diseño 1: Dashboard Principal MVP

The dashboard for the user '\$fedeverges' displays the following information:

- Pesos Disponibles (👁️):** \$1.240,75. A progress bar indicates the 'Presupuesto Mensual: 68% gastado'.
- Category Spending:**
 - 🛒 Súper: 49%
 - 🚗 Transp: 58%
 - 🎾 Tenis: 0%
- ÚLTIMOS GASTOS DEL MES:**
 - Supermercado** - \$15.000 (Hoy, 13:15 · Alimentos)
 - Cine Center** - \$8.500 (Ayer · Entretenimiento)
 - Pasaje Autobús** - \$12.000 (30 Mayo · Transporte)
- Actions:** A link 'Ver Todos los Movimientos →' and a floating action button (+) are at the bottom.

Diseño 2: Formulario de Entrada Rápida

The 'Nuevo Movimiento Local' form contains the following fields and options:

- Monto de la Operación:** \$ 15.000,00
- Concepto / Nota Corta:** Milanesas y verduras súper
- ASIGNACIÓN ESTRUCTURADA:**
 - Tipo:** GASTO
 - Categoría:** 🛒 Comida / Súper
 - Destino:** Efectivo Local
- Guardar Movimiento** (button)

4. Arquitectura de Almacenamiento Local (Esquema de Tablas)

Para asegurar la escalabilidad hacia la Fase 2, la persistencia local de la Fase 1 estructurará los datos utilizando un modelo relacional normalizado. Esto evitará migraciones destructivas de datos cuando se conecte el backend en la nube.

- **Tabla `Transactions`:** Almacena el registro atómico de cada movimiento. Campos core: `id` (UUID), `amount` (Decimal), `type` (ENUM: Gasto, Ingreso), `concept` (TEXT), `category\_id` (FK), `created\_at` (TIMESTAMP) y `installments\_total` (INTEGER para CuotaFix).
- **Tabla `Categories`:** Contiene el catálogo de clasificación de gastos manipulable por el usuario. Campos core: `id` (INT), `display\_name` (TEXT), `icon\_identifier` (TEXT) y `budget\_limit` (DECIMAL, opcional para alertas de control).
- **Tabla `FixedExpenses`:** Modulo derivado del benchmark. Guarda las plantillas de gastos recurrentes que el motor de la app debe instanciar automáticamente de forma local al cumplirse el ciclo de facturación establecido.

5. Estrategia de Despliegue para Play Store

El desarrollo multiplataforma nos permite empaquetar el código fuente en un artefacto compilado nativo de Android (`.apk` para pruebas internas y `.aab` para distribución oficial). Al no depender de servidores web externos en la Fase 1, el proceso de aprobación en la consola de desarrolladores de Google (Google Play Console) es sumamente ágil, al no requerir configuraciones complejas de endpoints de autenticación ni políticas de privacidad de datos de servidores de terceros.